

Módulo 2: Traducción del Modelo GAMS al Entorno DRL (Gymnasium)

Ingeniería de Inteligencia Artificial Financiera

Fase Cero - Proyecto de Tesis

1. Introducción: El Cambio de Paradigma

En la Clase 1 estudiamos al “Inversor Dios” (GAMS), un optimizador determinista que mira el tiempo futuro de golpe ($\sum_{t=1}^T$) para encontrar la política perfecta de pesos $w_{i,t}$ resolviendo una inmensa ecuación estática de Investigación de Operaciones.

Sin embargo, en el mundo real, los inversores no conocen el futuro. Operan bajo fuerte incertidumbre paso a paso. Para cruzar hacia el modelamiento de vanguardia, abandonaremos la resolución matemática simultánea y construiremos un **Agente Reactivo Autónomo** usando Reinforcement Learning (Algoritmos PPO y SAC).

Para lograr esto, encapsularemos el mercado financiero bajo el formalismo de un **Proceso de Decisión de Markov (MDP)** implementado en la librería estándar **Gymnasium**. Un MDP requiere traducir nuestra realidad a tres componentes sagrados: Estado (S_t), Acción (A_t) y Recompensa (R_t).

(Duda Fundamental: ¿Qué es Gymnasium y de dónde sacará los datos el Agente?)
Es vital aclarar que **Gymnasium NO** es una base de datos financiera. **Gymnasium** es simplemente un paquete de software (una “arquitectura de simulación” vacía creada inicialmente por OpenAI) que sirve de estándar mundial para programar videojuegos o entornos. Nosotros construiremos un entorno propio dentro de esta cáscara. ¿De dónde comerá los datos la Inteligencia Artificial? **De tus idénticos archivos crudos de GAMS (ps.gms y.gdx.xlsx)**. Escribiremos un script en Python que lea esos Excel, meta la historia a Gymnasium, y use el simulador para reproducirlos como una película paso a paso, forzando a PPO y SAC a sobrevivir en la misma historia bursátil exacta que GAMS procesó de golpe.

2. 0. Refresco de Memoria: ¿Qué es DRL y la Ecuación de Bellman?

Antes de ahondar en nuestro diseño exacto, recordemos que el ****Aprendizaje por Refuerzo Profundo (DRL)**** no mapea un input a un output como lo haría un clasificador genérico de Machine Learning. Aquí, redes neuronales gigantescas intentan aprender una **Política** probabilística (π) de supervivencia: “Dado cómo veo la bolsa de valores hoy, ¿qué botón aprieto para maximizar mi dinero en los próximos dos años?”

Para resolver esto, el algoritmo no solo debe observar la recompensa que recibe hoy (r_t), sino que debe adivinar telepáticamente el impacto a largo plazo de esa acción. Esto lo logra anclándose en la **Función de Valor de Estado (V^π)**, gobernada mundialmente por la **Ecuación de Bellman**:

$$V^\pi(s_t) = \mathbb{E}_{A_t \sim \pi} [r_t(s_t, A_t) + \gamma \mathbb{E}_{R_t} [V^\pi(S_{t+1})]] \quad (1)$$

¿Qué significa esto al español? Que el "Valor esperadoreal de estar sentado hoy en el estado de mercado s_t , es matemáticamente igual a: el premio inmediato que recibas por tu movimiento (r_t) **MÁS** la estimación futura de dónde quedarás parado el día de mañana ($V^\pi(S_{t+1})$). Ese futuro se multiplica por un factor de descuento γ (ej. 0.99) debido a que la plata del mañana vale menos hoy y la incertidumbre castiga el futurologismo. DRL simplemente entrena redes neuronales inmensas (El Crítico") para intentar resolver V^π , mientras simultáneamente otra red neuronal (El Actor") utiliza esos puntajes para decidir las acciones π .

Anatomía de las Redes: Actor (π) y Crítico (V / Q)

Una red neuronal en estos algoritmos combinados (tanto PPO como SAC) nunca está sola; hay dos módulos neuronales principales dialogando íntimamente en una arquitectura conocida como Actor-Crítico:

- **El Crítico:** Es tu tasador o asesor in-house. A este módulo le entran los datos bursátiles (s_t) y escupe el valor escalar predicho monetario. Conceptualmente manejan **las mismas variables matemáticas** V y Q que rigen a la Ecuación Ventaja ($A_t = Q - V$), pero cada algoritmo las construye arquitectónicamente *al revés*:
 - **PPO** entrena explícitamente a su Red Crítica para adivinar solo el valor global $V(s)$. Como no tiene una Red Q apartada, deduce "al vuelo" los valores $Q(s, a)$ usando la matemática temporal y las recompensas que vaya encontrando.
 - **SAC** toma el camino inverso. Entrena explícitamente a su Red para adivinar *Soft Q-Networks* $Q(s, a)$ (tasando la acción directamente y sumándole el premio de Entropía). Como no entrena explícitamente una red $V(s)$, la deduce por descarte de la esperanza de $Q(s, a)$. ¡Ambos algoritmos llegan a la misma Ventaja (A_t) por rumbos simétricos!

Cualquiera sea la vía, el Crítico nunca toma acciones ni invierte un peso. *Mecánicamente*, lo que la Red Neuronal del Crítico hace por dentro para "estimar" V o Q es recibir el vector de datos del mercado en su capa de Input, procesarlos en cascada mediante Multiplicación de Matrices y funciones de activación no lineales, y condensar todo en **un único nodo de salida** que arroja un número escalar continuo lineal (ej: 14,5\$). La red "aprende iterativamente aplicando una función de Loss por Error Cuadrático Medio (MSE), comparando el número escalar que la red adivinó hoy versus la Recompensa Real (Bootstrap) que acaba de observar temporalmente, empujando a los pesos de las matrices a no equivocarse de nuevo.

- **El Actor (La Política π):** *Ojo, la función π aplica idénticamente para PPO y SAC.* Ambos algoritmos dictan que esta red es tu gestor ejecutor. Toma el estado s_t , pero su última capa *no arroja un porcentaje de inversión literal plano*. La salida de esta red en ambos modelos es siempre **la parametrización de una distribución de probabilidad condicional** $\pi(a|s)$. Matemáticamente, la definición estricta es:

$$\pi(a_t | s_t) \doteq \mathbb{P}(A_t = a_t | S_t = s_t) \quad (2)$$

Esto se lee textualmente como: "La probabilidad de elegir tomar la acción de portafolio a_t , dado que el HMM me dice que estoy observando el estado bursátil s_t ". Como modelamos pesos fraccionales de un portafolio (espacio numérico continuo), el algoritmo asume que esta probabilidad obedece a una **Distribución Normal (Campana de Gauss)**. Por tanto, los nodos finales de la Red Neuronal Actor escupen una Media dictada en base al estado ($\mu_\theta(s_t)$)

y una Desviación Estándar ($\sigma_\theta(s_t)$), dando forma a la función de densidad paramétrica:

$$\pi_\theta(a_t | s_t) = \frac{1}{\sigma_\theta(s_t)\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{a_t - \mu_\theta(s_t)}{\sigma_\theta(s_t)}\right)^2\right) \quad (3)$$

De esa Función Gaussiana generada en vivo por la Red, el entorno Gymnasium extrae (samplea) la variable aleatoria bruta de inversión final. (La curiosidad central de SAC es que fuerza artificialmente a su Actor a engordar y ensanchar la métrica σ_θ valiéndose del factor entrópico, para mantenerse altamente audaz e impredecible).

La Función Ventaja (A_t : El Escondite del Reward Real)

En las fórmulas matemáticas asusta no encontrar el R_t numérico puro de forma explícita. Esto es porque un Reward crudo tiene demasiado ruido estocástico y confunde a la IA. El aprendizaje DRL contemporáneo se basa en la **Función Ventaja (Advantage)**: $A_t = Q(s_t, a_t) - V(s_t)$.

Para no dejar cabos sueltos teóricos en tu documento, la ecuación matemática pura para la Función de Acción-Valor Q^π se define como:

$$Q^\pi(s_t, a_t) \doteq \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s_t, A_t = a_t \right] \quad (4)$$

La cual, expresada coloquialmente en su forma recursiva de Bellman para calcular el salto inmediato, queda establecida como la sumatoria de la recompensa actual más el valor futuro descontado:

$$Q^\pi(s_t, a_t) \doteq \mathbb{E}_{S_{t+1}} [r_t(s_t, a_t) + \gamma V^\pi(S_{t+1})] \quad (5)$$

Teniendo la fórmula a la vista, notamos que A_t evalúa fríamente si la transacción específica seleccionada (Q) rindió sustancialmente *más* o *menos* ganancias que el promedio ponderado esperado del estado que calculamos al principio (V).

Si el A_t arroja saldo matemático positivo (.Acabas de minar oro que nadie vio venir en la bolsa”), el optimizador premiará al Actor aplicando Backpropagation sobre sus neuronas. Esto fuerza a la red a subir su Media Gaussiana (μ_θ) empujando a que esa masa fraccional de inversión sea altamente más probable que se repita la próxima vez. Si el A_t es negativo, la red aflojará todo purgando las chances de jamás volver a caer y hundir el portafolio.

¿Pero cómo conoce”la Ventaja si no sabe adivinar el futuro? (El Bootstrapping)

Una duda avanzada para tu tesis es: Si la Ventaja depende de estimaciones V hechas por una red neuronal tonta, ¿cómo sabemos que esa Ventaja es un número real al entrenar? La respuesta pura es el **Error de Diferencia Temporal (TD Error δ_t)**. Cuando el Agente ejecuta la acción a_t y avanza en Gymnasium al bloque s_{t+1} , el simulador le **cobra/paga la recompensa matemática inmediata y 100 % real** r_t . A partir de esa recompensa cruda y fresca que te entregó la bolsa de tu simulación, el algoritmo aproxima la Función Q y reemplaza la Ventaja por algo calculable:

$$A_t \approx \delta_t = \underbrace{r_t + \gamma V_\phi(s_{t+1})}_{\text{Lo que "Q"resultó ser}} - \underbrace{V_\phi(s_t)}_{\text{Lo que el Crítico esperaba}} \quad (6)$$

Revisa la hazaña de esta ecuación: la IA usa el Reward real empírico (r_t) que acabó de recolectar como ancla de la verdad humana, pero ”hace trampa”(Bootstrapping) e inyecta la propia **predicción del Crítico sobre el mañana** ($\gamma V(s_{t+1})$) para rellenar lo que falta del futuro lejano y ciego. Al principio del entrenamiento, el Crítico es estúpido y su estimación de la Ventaja es pura basura. Pero tras interactuar miles de veces con el Gymnasium, el Backpropagation va castigando al Crítico

usando su *Error Cuadrático Medio* frente a la sumatoria de las realidades r_t . A medida que el Crítico aprende a la fuerza a predecir finanzas correctamente, esa ecuación de Ventajas se aproxima velozmente a ser un reflejo perfecto del valor fundamental de tu portafolio.

Recordatorio Cero de los Titanes (PPO vs SAC)

En miras del modelamiento financiero, debes siempre recordar la principal filosofía de choque entre ambos titanes de Reinforcement Learning:

- **On-Policy (PPO):** Son algoritmos que aprenden “.en el aquí y ahora”. PPO juega la bolsa de valores un rato, recolecta datos frescos, calcula la Ventaja A_t , actualiza su Red Neuronal Actor, y luego **tira esa data a la basura**. Jamás entrena reciclando datos antiguos, requiere sangre nueva en cada simulación. Es estable y predecible.
- **Off-Policy (SAC):** Algoritmos con memoria fotográfica infinita (**Replay Buffer**). SAC guarda cada milisegundo o compraventa tabulada en un disco duro inmenso. Pasa la mayoría de su tiempo de entrenamiento recogiendo recuerdos aleatorios muertos del Buffer, cruzándolos con simulaciones vivas. Esta agitación estadística sumada a su adicción por la Entropía hace que descubra cosas estrafalarias.

3. 1. El Estado (S_t): Lo que la Inteligencia Artificial Observa

En un entorno *Gymnasium*, la función `step()` le entrega a la IA una fotografía del mundo actual antes de pedirle que tome una decisión. A esto lo llamamos el **Espacio de Observaciones**.

Basados estrictamente en el documento maestro de tu tesis, en la semana t , el sensor de nuestra Red Neuronal ingerirá un vector matricial (S_t) compuesto de:

$$S_t = \{x_t, w_{t-1}, \text{features}_t\} \quad (7)$$

Desglose para nuestro caso de 2 Activos (SPX y Cripto):

- x_t : **Capital Disponible Nominal**. (Ej. \$15,000 USD o 10.000). Le dice a la IA si estamos ganando o perdiendo plata acumulada y cual es nuestro poder de compra actual.
- w_{t-1} : **Pesos Anteriores (La Memoria de Fricción)**. Fundamental. Le pasamos qué porcentaje de la torta teníamos en SPX y Cripto la semana *pasada* (Cierre de $t-1$). Sin este dato histórico mínimo, la IA jamás podría saber si al moverse al nuevo w_t va a incurrir en costos por reacomodar su portafolio.
- features_t : **Las Señales del Mercado (El Oráculo)**. Aquí inyectamos el conocimiento algorítmico que vimos en la Clase 1:
 - Las probabilidades de régimen HMM para esta semana ($p_{\text{bull},t}, p_{\text{bear},t}$).
 - Las matrices de covarianza y varianza vigentes extraídas históricamente ($\Sigma_{i,j,t}$) y medias ($\mu_{i,t}$).
 - Volatilidades adicionales de mercado.

Punto Clave: GAMS leía estos inputs para *toda* la historia a la vez. Nuestra IA de PPO/SAC solo los leerá para el intervalo actual t ciegamente buscando anticipar el paso $t + 1$, forzándola a reaccionar bajo estrés y realidad temporal.

4. 2. El Espacio de Acciones (A_t): Lo que la IA Decide

Una vez que la red neuronal engulle el Estado S_t , sus múltiples interconexiones (Redes Multicapa - MlpPolicy) deben .escupir una decisión hacia el Entorno.

¿Qué acción debe tomar nuestro Agente? Exactamente lo definido como nuestra Variable Principal en GAMS: **Debe emitir el vector de nuevos pesos para nuestro dinero** (w_{SPX} y w_{Cripto}).

Sin embargo, enfrentamos un problema de diseño técnico en DRL: PPO o SAC emiten números reales continuos brutos $\tilde{a}_t$, lo que significa que la IA podría inútilmente decidir .asignar -40 % de peso al SPX y 1900 % a Cripto”, violando nuestras leyes físicas.

La Solución (La Proyección al Símpex):

Para obligar matemáticamente a la salida neuronal a comportarse como un portafolio factible (Restricciones 1 y 2 de presupuesto de GAMS sin escribir código duro), el entorno tomará la señal bruta de la IA ($\tilde{a}_t$) y la atravesará por una capa de limitación funcional:

$$w_{i,t} = \frac{\max(0, \tilde{a}_{i,t})}{\sum_{j \in I} \max(0, \tilde{a}_{j,t})} \quad (8)$$

Impacto: Cualquier locura numérica que escupa la IA se volverá positiva por el $\max(0)$ y quedará fraccionada respecto a su propio total, asegurándose matemáticamente de que la suma perfecta de todo sea **1,0** (100 % del capital asignado) y sin nada de ventas en corto.

5. 3. El Proceso de Retorno y la Dinámica de Transición

Antes de calcular cualquier ganancia o castigo, nuestro entorno Gymnasium debe materializar lo que ocurrió en el mercado bursátil tras mover los fondos. Basándonos estrictamente en la formulación de tu documento maestro, la física del MDP dicta esto:

El Proceso de Retornos (R_t)

Consumada la elección w_t , el entorno hace avanzar la línea cronológica y extrae el vector de retornos reales que sufrieron los activos $R_t = (R_{i,t})_{i \in I}$. Este vector obedece centralmente a una distribución estocástica $\mathcal{D}_t$:

$$R_t \sim \mathcal{D}_t(\mu_t, \Sigma_t) \quad (9)$$

La gran clave de esto es que esa **distribución** $\mathcal{D}_t$ en nuestra tesis está *condicionada y gobernada mediante una mezcla de regímenes dictados por el modelo HMM (bear o bull)*. Esto significa que sus parámetros μ_t y Σ_t no son absolutos; son tensores dinámicos condicionados (*mumix* y *sigma-mix* en GAMS) porque el entorno transita estructuralmente entre periodos base de euforia o alta volatilidad inflacionaria.

La Dinámica de la Transición ($\mathcal{P}$ y Estado Futuro)

Con la acción impuesta $a_t = w_t$ y un retorno bursátil observado aleatorio R_t , Gymnasium decreta que todo tu capital avanza ciegamente al siguiente bloque temporal $S_{t+1} = \mathcal{F}(S_t, A_t, R_t)$ calculando tres grandes métricas ineludibles:

1. **El Retorno Puro del Portafolio** ($rport_t$): Es la suma ponderada transversal del retorno individual ($R_{i,t}$) de cada activo multiplicado por el porcentaje de tu dinero alojado ahí ($w_{i,t}$):

$$rport_t = \sum_{i \in I} w_{i,t} R_{i,t} = w_t^\top R_t$$

2. **Salto Evolutivo del Dinero (Capital x_{t+1}):** El capital de flotación para mañana queda estrictamente dictaminado por tu capital original inflado (o mermado) por $w_t^\top R_t$, despintando rigurosa y dolorosamente toda la comisión porcentual de fricción por rebalanceo de tu broker c_i :

$$x_{t+1} = x_t \left(1 + w_t^\top R_t - \sum_{i \in I} c_i |w_{i,t} - w_{i,t-1}| \right) \quad (10)$$

3. **Memoria Viscosa:** Tu recién elegido w_t pasa a la historia. Se encasillará forzosamente como el w_{t-1} (los pesos vigentes al cierre) en el paso de mañana, anclando al modelo a tener la carga temporal (costos) en caso de sufrir un ataque de pánico y cambiar sus convicciones fuertemente luego.

6. 4. La Recompensa (r_t): El Sistema de Dopamina

La piedra angular del Aprendizaje por Refuerzo. Ni PPO ni SAC saben de finanzas. Ellos son redes neuronales herméticas que entrenan ajustando probabilidades de Gauss (Backpropagation) guiándose única y exclusivamente por el puntaje de evaluación (Reward) que Gymnasium les emita.

Es aquí donde ocurre la majestuosa fusión de tu tesis: **Tomaremos la vieja Función Objetivo Z determinista del Módulo 1 en GAMS, y la convertiremos oficialmente en el Sistema de Premio-Castigo de la IA simulada:**

$$r_t(s_t, a_t) = \left(\sum_i w_{i,t} \mu_{i,t} \right) - \lambda \left(\sum_{i,j} w_{i,t} w_{j,t} \Sigma_{i,j,t} \right) - \left(\sum_i c_i |w_{i,t} - w_{i,t-1}| \right) \quad (11)$$

Si el agente toma una decisión w_t agresivamente arriesgada frente a una matriz Σ cargada por p_bear , recibirá en su cara una **Recompensa brutalmente negativa**. Sufrir y adapta sus pesos para no hacerlo nunca más.

Nota Avanzada: A diferencia del modelo estático en GAMS, la penalización de comercio (último término matemático) ahora sí puede calcularse directamente midiendo el Valor Absoluto real ($|w_{i,t} - w_{i,t-1}|$). Deep Reinforcement Learning opera bajo ensayo y error por pasos en un simulador (Gymnasium), a diferencia de la optimización lineal que colapsa con no-linealidades, nuestra IA soporta calcular diferencias absolutas fácilmente.

Día a día, episodio tras episodio, la herramienta irá recorriendo nuestra base de datos.

7. 5. Modelos en Batalla: ¿Cómo tragan PPO y SAC esta Fórmula?

Aunque tu documento de tesis dicte que ambos algoritmos reciben el mismo MDP exacto (Mismo Entorno, mismo S_t, A_t, R_t), su cerebro neuronal lo procesa y optimiza de formas diametralmente opuestas. Por eso vamos a poner a competir en ring:

1. Proximal Policy Optimization (PPO - El Metódico Seguro):

PPO es un algoritmo *On-Policy*. Juega un set de datos, calcula la Ecuación de Bellman temporal, y luego actualiza su política π . Su sello distintivo no modifica la recompensa, sino **cómo aprende matemáticamente**: usa una función objetivo llamada **Clipping Surrogate**.

$$J^{CLIP}(\theta) = \mathbb{E} [\text{mín}(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (12)$$

Donde la división $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ es llamado el **Ratio Probabilístico**. Es un radar de qué tanto quiere cambiar su política actual la Red Neuronal vs lo que creía correcto en su versión pasada: si este da $\gg 1,0$, el Actor está intentando de forma súbita inyectar masivas probabilidades a favor de esa acción en el sistema.

*¿Qué magia hace el **Clipping (El Fusible)** dentro de J ?*

Si el Agente descubre sorpresivamente que la acción de pasarse por completo a activos Cripto arrojó Ventajas históricas multimillonarias (A_t colosalmente positiva), un algoritmo de gradiente clásico arrojaría un **Salto de Gradiente Catastrófico**, es decir, tiraría toda su historia formativa a la basura para re-cablear su cerebro violenta y desproporcionadamente hacia criptomonedas, rompiendo la matemática de años de pre-entrenamiento.

PPO detiene esto con el efecto restrictivo del `clip(..., 1- $\epsilon$ , 1+ $\epsilon$ )` con la frontera estrecha del $\pm 20\%$ ($0,8 \rightarrow 1,2$). Existen 3 escenarios durante el Backpropagation algorítmico:

1. **Desarrollo Monótono Normal:** El Actor sugiere desviar sus intenciones solo un ligero 4% extra hacia SPX ($r_t = 1,04$). Esta cifra vive cómodamente bajo el 1.2 normativo por lo que el `clip` no le estorba. El Gradiente de cálculo fluye perfecto y la red neuronal fortalece su sapiencia de forma fina y orgánica.
2. **La Codicia Descontrolada:** La acción rindió un pozo de oro monumental ($A_t \gg 0$) y en su euforia artificial, el Actor Neural quiere multiplicar un 500% la tendencia a tomar tal acción ($r_t = 5,0$). Aquí, la frontera `clip` le amputa de golpe todo lo sobrante, aplanando la curva J^{CLIP} horizontalmente contra el techo 1,2. La parte central de este mecanismo es que **cuando la curva objetivo se aplanan, la derivada natural cae y el Gradiente se vuelve CERO** ($\nabla J = 0$). Al forzar el gradiente a la nulidad estática, los pesos de conexión intra-neuronales del algoritmo **frenan en seco su actualización**. Este parón matemático le prohíbe apostar todo a lo que podría constituir pura varianza suertuda, empujando la actualización hacia la paciencia conservadora.
3. **El Pánico Absoluto:** Cripto colapsa y la Red quiere purgar masiva e instintivamente la acción bajando el ratio al subsuelo. PPO repite el patrón de corte bajista al piso 0.8, apagando la actualización $\nabla J_{old} = 0$, salvándole la vida a la curva de aprendizaje de forma metódica.

Esta trinidad garantiza que PPO escale las laderas probabilísticas de portafolios hiper-volátiles con pasos de pajarito con certidumbre monótona. Es el inversor de librito, la barrera contra el caos, pero a costa de mucha de la creatividad fuera de la caja que el SAC entropizado de tu tesis podría presentar en valles no explorados.

2. Soft Actor-Critic (SAC - El Explorador Múltiple):

SAC es el modelo **Off-Policy** por excelencia e implementa el estado del arte: *Maximum Entropy RL*. SAC no solo quiere ganar mucha plata (maximizar el R_t normal de Markowitz), sino que la arquitectura le **suma a escondidas un inmenso bono de recompensa por ser errático y aleatorio (Entropía $\mathcal{H}$)** en su propia función matemática principal:

$$J(\pi) = \sum_t \mathbb{E} [r_t(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))] \quad (13)$$

La ecuación arriba te revela un secreto de diseño de tu tesis: la Red de SAC cree que su deber es optimizar tu Reward Markowitz (r_t) **MÁS** un empuje extra de entropía proporcional a α . Esto le inyecta literalmente dopamina a la neurona por probar acciones estúpidas o no convencionales en vez de encasillarse en lo que ya conoce. Al obligarlo a mantener alta entropía, SAC suele descubrir

senderos rarísimos de ganancia a través de HMM y varianzas que PPO, en su infinita seguridad, jamás se atrevería a explorar.

Para tu tesis, esto es enfrentar a un Inversor Conservador de librito (PPO) contra un Inversor Algorítmico Probabilista fuera de la caja (SAC) usando exactamente las mismas reglas de GAMS.